

Entrega LAB1

1.Chat with buddies

1.1. The server

Tenim com objectiu mantenir la llista de clients connectats i enviar a tots els clients els esdeveniments que poden passar en qualsevol moment al chat (join, leave, message).

Primer de tot, declarem el mòdul i exportem la API pública start/0.

```
-module(server).  
%% Exported Functions  
-export([start/0]).
```

Seguidament, ja podem començar amb el bloc de funcions de la API:

- **start()** → tenim spawn() crea el procés servidor començant el process_requests() amb cap client connectat (estat inicial)
- **register()** → publica server com procés amb nom 'myserver' per a que els futurs clients puguin connectar-se amb aquest nom.
- **process_requests(Clients)** → és el bucle reactiu amb el llistat dels clients connectats
 - **client_join_req** → es dona d'alta un client. Utilitzem el From (PID facilitat pel client) per a que el server pugui enviar-li esdeveniments. Tots els clients reben els missatges (broadcast). Finalment, s'actualitza la llista de clients amb el nou client connectat.
 - **client_leave_req** → eliminem el PID facilitat pel client (From) de la llista amb delete. Després enviem un missatge a tots els clients (ell inclós) de que marxa i enviem la senyal exit al client per a que el background process finalitzi.
 - **send** → enviem el missatge rebut per un usuari a tots el clients connectats amb el nom d'aquest i el seu text.
 - **disconnect** → desregistrar el nom

```

process_requests(Clients) →
  receive
    {client_join_req, Name, From} →
      NewClients = [From|Clients], %% TODO: COMPLETE
      broadcast(NewClients, {join, Name}),
      process_requests(NewClients); %% TODO: COMPLETE
    {client_leave_req, Name, From} →
      NewClients = lists:delete(From, Clients), %% TODO: COMPLETE
      broadcast(Clients, {leave, Name}),
      From ! exit,
      process_requests(NewClients); %% TODO: COMPLETE
    {send, Name, Text} →
      broadcast(Clients, {message, Name, Text}),
      process_requests(Clients);
  disconnect →
    unregister(myserver)
end.

```

Finalment, amb les funcions locals enviem asíncronament missatges a cada PID, amb ! posem el missatge a la bústia del receptor.

```

broadcast(PeerList, Message) →
  Fun = fun(Peer) → Peer ! Message end,
  lists:map(Fun, PeerList).

```

Ens podem fixar com es realitza una **recursió de cua** en tot moment amb process_requests(), ja que cada vegada que processem un missatge tornem a cridar a la funció amb l'estat actualitzat de la llista de Clients.

1.2. The client

Tenim el procés background per rebre i mostrar missatges: join, leave i exit per acabar. També tenim el procés principal per llegir del teclat i enviar send i client_leave_req.

Al principi, tenim el mòdul del client amb start com a punt de partida, creant el procés background amb spawn() per gestionar les respostes del servidor. Seguidament, el procés principal ja entra a process_commands() amb el PID del background.

```

-module(client).
%% Exported Functions
-export([start/2]).

```

En segon lloc, tenim el init_client() on podem veure com el procés background es registra amb el servidor amb l'enviament del seu PID amb self() quedant a l'espera dels esdeveniments amb la comanda ! (possa el missatge a la bústia del server).

```
init_client(ServerPid, MyName) →
    ServerPid ! {client_join_req, MyName, self()},
    process_requests().
```

En tercer lloc, ja tenim les funcions locals que gestiona el background. Primer amb join que quan un usuari entra, el procés imprimeix el missatge d'avís amb el nom de l'usuari que ha entrat i segueix executant el bucle. Després tenim el leave que avisa amb el nom de l'usuari que ha abandonat. A continuació, quan arriba un missatge surt per pantalla amb el [Name] de l'emissor. Finalment, quan rep exit pel servidor finalitza el procés background.

```
process_requests() →
    receive
    {join, Name} →
        io:format("[JOIN] ~s joined the chat~n", [Name]),
        process_requests();
    {leave, Name} →
        io:format("[LEAVE] ~s left the chat~n", [Name]),
        process_requests();
    {message, Name, Text} →
        io:format("[~s] ~s", [Name, Text]),
        process_requests();
    exit →
        ok
    end.
```

En quart, i últim lloc, tenim el procés principal. El seu funcionament és el següent, llegeix una línia de teclat:

- Si llegeix **exit** → demana al servidor un leave request per abandonar.
- Si llegeix **qualsevol altre missatge** l'envia al server i fa recursió de cua cridant-se a si mateix per llegir la següent línia de teclat.

```
process_commands(ServerPid, MyName, ClientPid) →
    %% Read from standard input and send to server
    Text = io:get_line("→ "),
    if
        Text == "exit\n" →
            ServerPid ! {client_leave_req, MyName, ClientPid},
            ok;
        true →
            ServerPid ! {send, MyName, Text},
            process_commands(ServerPid, MyName, ClientPid)
    end.
```

1.3. Testing

Podem veure un exemple pràctic del que hem creat amb dos usuaris parlant desde el mateix servidor.

```
[(base) hugohf05@MacBook-Air-de-Hugo-3 chatty-source-files % erl -name server_node@127.0.0.1 -setcookie secret
Erlang/OTP 28 [erts-16.1] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-threads:1] [jit] [dtrace]

Eshell V16.1 (press Ctrl+G to abort, type help(). for help)
(server_node@127.0.0.1)1> c(server).
{ok,server}
[(server_node@127.0.0.1)2> server:start().
true
[(server_node@127.0.0.1)3> ]
```

```
chatty-source-files -- beam.smp -- -root /opt/homebrew/Cellar/erlang/28.1/lib/erlang -bindir /opt/homebrew/Cellar/erl...
(1)oaded (v)ersion (k)ill (D)b-tables (d)istribution
a
(base) hugohf85@MacBook-Air-de-Hugo-3 chatty-source-files % erl -name client_node@127.0.0.1 -setcookie secret
Protocol 'inet_tcp': the name client_node@127.0.0.1 seems to be in use by another Erlang node
(base) hugohf85@MacBook-Air-de-Hugo-3 chatty-source-files % erl -name client_node2@127.0.0.1 -setcookie secret
Erlang/OTP 28 [erts-16.1] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-threads:1] [jit] [dtrace]

Eshell V16.1 (press Ctrl+G to abort, type help(). for help)
(client_node2@127.0.0.1)> c(client).
{ok,client}
(client_node2@127.0.0.1)> client:start({myserver, 'server_node@127.0.0.1'}, "Hugo").
[JOIN] Hugo joined the chat
Command is taking a long time, type Ctrl+G, then enter 'i' to interrupt
-> hola david
[Hugo] hola david
[David] hola hugo
-> como estas
[Hugo] como estas
[David] muy bien y tu?
-> perfectamente
[Hugo] perfectamente
->

a
(base) hugohf85@MacBook-Air-de-Hugo-3 chatty-source-files % erl -name client_node@127.0.0.1 -setcookie secret
Erlang/OTP 28 [erts-16.1] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-threads:1] [jit] [dtrace]

Eshell V16.1 (press Ctrl+G to abort, type help(). for help)
(client_node@127.0.0.1)> c(client).
{ok,client}
(client_node@127.0.0.1)> client:start({myserver, 'server_node@127.0.0.1'}, "David").
[JOIN] David joined the chat
Command is taking a long time, type Ctrl+G, then enter 'i' to interrupt
-> hola hugo
[David] hola hugo
[JOIN] Hugo joined the chat
[Hugo] hola david
-> hola hugo
[David] hola hugo
[Hugo] como estas
-> muy bien y tu?
[David] muy bien y tu?
[Hugo] perfectamente
->
```

1.4. Open Questions

Q1 - Does this solution scale when the number of users increase?

No podem escalar molt perquè és un únic servidor centralitzat amb problemes futurs de fallades i coll d'ampolla quan volem incrementar el nombre d'usuaris al sistema.

Q2 - What happens if the server fails?

Com tots els clients són dependents d'un únic servidor, cap d'ells podrà xatejar i quedarà un sistema inútil.

Q3 - Are the messages from a single client guaranteed to be delivered to any other client in the order they were issued?

Sí, podem garantir que els missatges es difonen amb el mateix ordre que han sigut enviats a tots els clients gràcies a que tenim un servidor seqüencial processant una petició darrera d'altre. Amb Erlang un emissor únic enviant a un únic receptor tenim FIFO per la recepció dels paquets/missatges.

Q4 - Are the messages sent concurrently by several clients guaranteed to be delivered to any other client in the order they were issued?

En aquest cas no estrictament, ja que el servidor tindrà l'ordre general en base a l'ordre d'arribada dels missatges a ell. Llavors poden no coincidir els temps d'emissió reals dels clients amb el temps d'enviament del servidor. El que sí que garanteix és la consistència de la visualització de l'ordre dels missatges als clients pel xat, tots veuran el mateix.

Q5 - Is it possible that a client receives a response to a message from another client before receiving the original message from a third client?

No, com bé hem comentat anteriorment tenim un únic servidor centralitzat seqüencial, és a dir, el mateix servidor rep el missatge original i la resposta en ordre de temps d'emissió. Per tant, el servidor envia als clients els esdeveniments en servei seqüencial mantenint l'ordre com a conseqüència de l'utilització de FIFO a la seva cua de missatges.

Q6 - If a user joins or leaves while the server is broadcasting a message, will he/she receive that message?

Tot dependrà de l'ordre d'arribada a la llista de missatges del servidor, ja que d'allà dependrà l'ordre de processament dels esdeveniments.

Si el servidor processa abans el broadcast del missatge que el leave, l'usuari rebrà el missatge. En cas contrari, no podrà veure el missatge.

Si el servidor processa abans el broadcast del missatge que el join, l'usuari no rebrà cap missatge. En cas contrari, ho podrà veure.

El servidor sempre difon el missatge a la llista de PIDs presents a la llista "Clients" al moment exacte que es crida a la funció broadcast(). Si el servidor ja està fent broadcast del missatge, llavors atén després els esdeveniments de entrar/surtir que l'enviament del missatge perquè no es troben a la llista de Clients, Erlang és seqüencial.

2.Making it robust

Amb aquesta nova versió podem tenir diversos servidors on cada client es connecta a qualsevol d'aquests però els esdeveniments (join,leave,message) de qualsevol d'ells apareixen a tots els servidors gràcies a el reenviament per broadcast que hem programat amb erlang al codi server2.erl (el codi client.erl romandrà el mateix per aquesta versió 2).

En primer lloc, tenim la inicialització dels servidors. Primer creem un servidor amb un conjunt de servidors on només hi és ell amb init_server(). Després creem un segon procés init_server(BootServer) per fer una petició server_join_req de unió al conjunt de servidors.

```
start() →
    ServerPid = spawn(fun() → init_server() end),
    register(myserver, ServerPid).

start(BootServer) →
    ServerPid = spawn(fun() → init_server(BootServer) end),
    register(myserver, ServerPid).

init_server() →
    %% Arranca como grupo con un solo servidor: uno mismo
    process_requests([], [self()]).

init_server(BootServer) →
    %% Pide unirse a un grupo existente
    BootServer ! {server_join_req, self()},
    process_requests([], []).
```

En segon lloc, realitzem la gestió de clients amb el bucle reactiu process_requests(Clients,Servers). Tenim diverses opcions de missatges a rebre:

- **client_join_req** → afegim un nou client a la llista de clients i difon als servidors (no als clients).
- **client_leave_req** → eliminem un client, difon als servidors i envia exit al client per a que marxi.
- **send** → difon als servidors el missatge quan un client envia text

Ens podem fixar com tenim un multicast lògic on els esdeveniments no s'emeten directament als clients, sinó que entre els servidors per després cada server fer un relay als seus clients locals.

```
%% Messages between client and server
{client_join_req, Name, From} →
    %% Añadimos el proceso cliente local a la lista
    NewClients = [From|Clients], %% TODO: COMPLETE

    %% Difundimos a TODOS los servidores (incluido uno mismo si está en Servers)
    broadcast(Servers, {join, Name}), %% TODO: COMPLETE
    process_requests(NewClients, Servers); %% TODO: COMPLETE

{client_leave_req, Name, From} →
    %% Quitamos el cliente de la lista local
    NewClients = lists:delete(From, Clients), %% TODO: COMPLETE

    %% Anunciamos el leave al conjunto de servidores
    broadcast(Servers, {leave, Name}), %% TODO: COMPLETE

    %% Avisamos al proceso cliente para que termine su listener
    From ! exit,
    process_requests(NewClients, Servers); %% TODO: COMPLETE

{send, Name, Text} →
    %% Un cliente local envía un mensaje: lo difundimos a todos los servidores
    broadcast(Servers, {message, Name, Text}), %% TODO: COMPLETE
    process_requests(Clients, Servers);
```

En tercer lloc, tenim els missatges entre els servidors:

- **disconnect** → informem a la resta de servidors quan un server es desconnecta/se'n va.
- **server_join_req** → afegim el nou servidor a la llista de Servers i avisem a la resta la nova incorporació.
- **update_servers** → actualitzem una còpia local dels servidors.

```

%% Messages between servers
disconnect →
    %% Este servidor se va: informamos al resto del nuevo conjunto
    NewServers = lists:delete(self(), Servers), %% TODO: COMPLETE
    broadcast(NewServers, {update_servers, NewServers}), %% TODO: COMPLETE
    unregister(myserver);
{server_join_req, From} →
    %% Llega un nuevo servidor: lo añadimos y avisamos a todos
    NewServers = [From|Servers], %% TODO: COMPLETE
    broadcast(NewServers, {update_servers, NewServers}), %% TODO: COMPLETE
    process_requests(Clients, NewServers); %% TODO: COMPLETE
{update_servers, NewServers} →
    io:format("[SERVER UPDATE] ~w~n", [NewServers]),
    process_requests(Clients, NewServers); %% TODO: COMPLETE

RelayMessage → %% Whatever other message is relayed to its clients
    broadcast(Clients, RelayMessage),
    process_requests(Clients, Servers)

```

En quart lloc, si arriba un missatge del client diferent als anteriors l'enviem únicament als clients locals amb RelayMessage.

```

RelayMessage → %% Whatever other message is relayed to its clients
    broadcast(Clients, RelayMessage),
    process_requests(Clients, Servers)

```

Testing

Podem veure un exemple pràctic del que hem creat amb dos usuaris parlant desde dos servidors diferents però connectats.

```

(base) hugohf05@MacBook-Air-de-Hugo-3 chatty-source-files % erl -name server_node@127.0.0.1 -setcookie secret

Erlang/OTP 28 [erts-16.1] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-threads:1] [jit] [dtrace]

Eshell V16.1 (press Ctrl+G to abort, type help(). for help)
(server_node@127.0.0.1)1> c(server2).
{ok,server2}
(server_node@127.0.0.1)2> server2:start().
true
[SERVER UPDATE] [<15628.97.0>,<0.97.0>]
(server_node@127.0.0.1)3> █

chatty-source-files — beam.smp -- -root /opt/homebrew/Cellar/erlang/28.1/lib/erlang -bindir /opt/homebrew/C.
(base) hugohf05@MacBook-Air-de-Hugo-3 chatty-source-files % erl -name server_node2@127.0.0.1 -setcookie secret

Erlang/OTP 28 [erts-16.1] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-threads:1] [jit] [dtrace]

Eshell V16.1 (press Ctrl+G to abort, type help(). for help)
(server_node2@127.0.0.1)1> c(server2).
{ok,server2}
(server_node2@127.0.0.1)2> server2:start({myserver,'server_node@127.0.0.1'}).
true
[SERVER UPDATE] [<0.97.0>,<15564.97.0>]
(server_node2@127.0.0.1)3> █

```



```
chatty-source-files — beam.smp -- -root /opt/homebrew/Cellar/erlang/28.1/lib/erlang -bindir /opt/homebrew/Cell...
[LEAVE] David left the chat
(client_node@127.0.0.1)3>
BREAK: (a)bort (A)bort with dump (c)ontinue (p)roc info (i)nfo
(l)oaded (v)ersion (k)ill (D)b-tables (d)istribution
a
(base) hugohf05@MacBook-Air-de-Hugo-3 chatty-source-files % erl -name client_node@127.0.0.1 -setcookie secret
Erlang/OTP 28 [erts-16.1] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-threads:1] [jit] [dtrace]

Eshell V16.1 (press Ctrl+G to abort, type help(). for help)
(client_node@127.0.0.1)1> c(client).
{ok,client}
(client_node@127.0.0.1)2> client:start({myserver,'server_node@127.0.0.1'}, "Hugo").
[JOIN] Hugo joined the chat
Command is taking a long time, type Ctrl+G, then enter 'i' to interrupt
[JOIN] David joined the chat
-> Hola David
[Hugo] Hola David
[David] que tal hugo
-> bien y tu?
[Hugo] bien y tu?
[David] muy bien , probando el servidor 2
-> yo probando el servidor 1
[Hugo] yo probando el servidor 1
->

chatty-source-files — beam.smp -- -root /opt/homebrew/Cellar/erlang/28.1/lib/erlang -bindir /opt/homebrew/Cella...
[Hugo] perfectamente
-> exit
ok
[LEAVE] Hugo left the chat
(client_node2@127.0.0.1)3>
BREAK: (a)bort (A)bort with dump (c)ontinue (p)roc info (i)nfo
(l)oaded (v)ersion (k)ill (D)b-tables (d)istribution
a
(base) hugohf05@MacBook-Air-de-Hugo-3 chatty-source-files % erl -name client_node2@127.0.0.1 -setcookie secret
Erlang/OTP 28 [erts-16.1] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-threads:1] [jit] [dtrace]

Eshell V16.1 (press Ctrl+G to abort, type help(). for help)
(client_node2@127.0.0.1)1> client:start({myserver,'server_node2@127.0.0.1'}, "David").
[JOIN] David joined the chat
Command is taking a long time, type Ctrl+G, then enter 'i' to interrupt
[Hugo] Hola David
-> que tal hugo
[David] que tal hugo
[Hugo] bien y tu?
-> muy bien , probando el servidor 2
[David] muy bien , probando el servidor 2
[Hugo] yo probando el servidor 1
```

A continuació podem veure un exemple on apareix les notificacions d'entrada i sortida d'un client que pot veure l'altre client de altre servidor al mateix chat comú.

```
(base) hugohf05@MacBook-Air-de-Hugo-3 chatty-source-files % erl -name client_node@127.0.0.1 -setcookie secret
Erlang/OTP 28 [erts-16.1] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-threads:1] [jit] [dtrace]

Eshell V16.1 (press Ctrl+G to abort, type help(). for help)
(client_node@127.0.0.1)1> c(client).
{ok,client}
(client_node@127.0.0.1)2> client:start({myserver,'server_node@127.0.0.1'}, "Hugo").
[JOIN] Hugo joined the chat
-> hola
[Hugo] hola
Command is taking a long time, type Ctrl+G, then enter 'i' to interrupt
-> me voy a ir
[Hugo] me voy a ir
-> exit
ok
(client_node@127.0.0.1)3>

chatty-source-files — beam.smp -- -root /opt/homebrew/Cellar/erlang/28.1/lib/erlang -bindir /opt/homebrew/Ce...
(client_node2@127.0.0.1)3>
BREAK: (a)bort (A)bort with dump (c)ontinue (p)roc info (i)nfo
(l)oaded (v)ersion (k)ill (D)b-tables (d)istribution
a
(base) hugohf05@MacBook-Air-de-Hugo-3 chatty-source-files % erl -name client_node2@127.0.0.1 -setcookie secret
Erlang/OTP 28 [erts-16.1] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-threads:1] [jit] [dtrace]

Eshell V16.1 (press Ctrl+G to abort, type help(). for help)
(client_node2@127.0.0.1)1> client:start({myserver,'server_node2@127.0.0.1'}, "David").
[JOIN] David joined the chat
Command is taking a long time, type Ctrl+G, then enter 'i' to interrupt
[Hugo] Hola David
-> que tal hugo
[David] que tal hugo
[Hugo] bien y tu?
-> muy bien , probando el servidor 2
[David] muy bien , probando el servidor 2
[Hugo] yo probando el servidor 1
[JOIN] Hugo joined the chat
[Hugo] hola
[Hugo] me voy a ir
[LEAVE] Hugo left the chat
```


Crash del server 2

Si un server cau, els seus clients locals perden la comunicació amb el sistema, però la resta de servers continua vigent amb normalitat sense cap detecció automàtica a la caiguda, és a dir, que el server caigut encara apareix a la llista de de Servers.

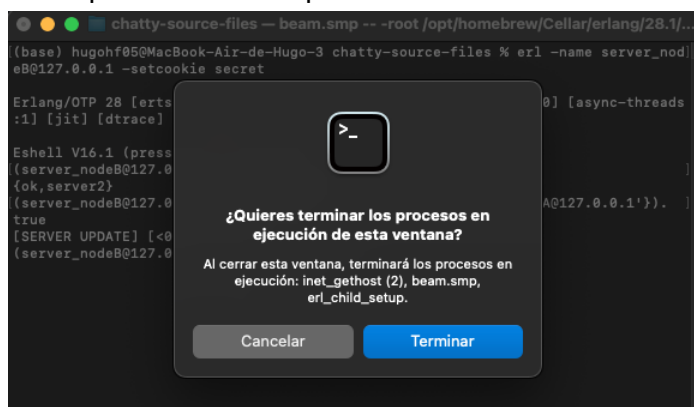
A continuació podem veure els dos servidors (A i B) creats correctament amb un client cadascú (David → A, Hugo → B) que han pogut comunicar-se sense problemes pel xat.

The image displays four terminal windows arranged in a 2x2 grid, showing the setup and operation of two servers (A and B) and two clients (David and Hugo).

- Top-left window:** Shows the setup of server A. The user runs `erl -name server_nodeA@127.0.0.1 -setcookie secret` and then `server2:start()`. The output shows the server is running on port 15628.
- Top-right window:** Shows the setup of server B. The user runs `erl -name server_nodeB@127.0.0.1 -setcookie secret` and then `server2:start()`. The output shows the server is running on port 15564.
- Bottom-left window:** Shows the setup of client David. The user runs `erl -name client_David@127.0.0.1 -setcookie secret` and then `client:start()`. The output shows the client is running on port 15628.
- Bottom-right window:** Shows the setup of client Hugo. The user runs `erl -name client_hugo@127.0.0.1 -setcookie secret` and then `client:start()`. The output shows the client is running on port 15564.

The bottom two windows also show chat messages being exchanged between the clients and servers, indicating successful communication.

Per tal de provocar el crash, tancarem la finestra del server B sense fer disconnect, matant així el procés beam.smp:



Enviem missatges desde qualsevol dels clients per veure si encara tenen comunicació i veiem que no. El client David (connectat al server A) pot continuar enviant missatges pel xat però Hugo (desconnectat pel server B) no te connexió amb el xat.

```
chatty-source-files -- beam.smp -- -root /opt/homebrew/Cellar/erlang/28.1/...
((base) hugohf05@MacBook-Air-de-Hugo-3 chatty-source-files % erl -name client_David@127.0.0.1 -setcookie secret

Erlang/OTP 28 [erts-16.1] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-threads:1] [jit] [dtrace]

Eshell V16.1 (press Ctrl+G to abort, type help(). for help)
(client_David@127.0.0.1)1> c(client).
(ok,client)
(client_David@127.0.0.1)2> client:start({myserver,'server_nodeA@127.0.0.1'}, "David").
[JOIN] David joined the chat
Command is taking a long time, type Ctrl+G, then enter 'i' to interrupt
[JOIN] Hugo joined the chat
-> Hola Hugo, estoy en el servidor A
[David] Hola Hugo, estoy en el servidor A
[Hugo] Hola David, estoy en el servidor B
[JOIN] Hugo joined the chat
[Hugo] Hola David, estoy en el servidor B
-> sigues ahí Hugo?
[David] sigues ahí Hugo?
->

chatty-source-files -- beam.smp -- -root /opt/homebrew/Cellar/erlang/28.1/...
((base) hugohf05@MacBook-Air-de-Hugo-3 chatty-source-files % erl -name client_hugo@127.0.0.1 -setcookie secret

Erlang/OTP 28 [erts-16.1] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-threads:1] [jit] [dtrace]

Eshell V16.1 (press Ctrl+G to abort, type help(). for help)
(client_hugo@127.0.0.1)1> c(client).
(ok,client)
(client_hugo@127.0.0.1)2> client:start({myserver,'server_nodeB@127.0.0.1'}, "Hugo").
[JOIN] Hugo joined the chat
Command is taking a long time, type Ctrl+G, then enter 'i' to interrupt
-> Hola David, estoy en el servidor B
[Hugo] Hola David, estoy en el servidor B
-> probando después de la caída de mi server
->
```

Per realitzar una sortida exitosa d'un server hem de realitzar un: disconnect.

```
(server_nodeA@127.0.0.1)3> disconnect.
disconnect
```

Open Questions

Q1 - What happens if a server fails?

Quan un servidor cau tots els seus clients connectats a aquest servidor ja no podran enviar ni rebre missatges o esdeveniments. Però tenim l'avantatge de que els altres servidors amb els seus clients continuen funcionant amb normalitat, mantenen una còpia de l'estat del xat. L'inconvenient que tenim és que la resta de Servers no es pot actualitzar automàticament després de la caiguda del Server, llavors no la detecten i poden continuar enviant esdeveniments al server caigut.

Com hem vist a teoria, es tracta d'una replicació no fiable, millorant la disponibilitat parcial sense detecció de caigudes ni recuperacions automàtiques (tolerància a fallades estàtica no dinàmica).

Q2 - Do your answers to previous questions iii, iv, and v still hold in this implementation?

3 → Els missatges continuen enviant-se ordenadament dins del server local, a Erlang un procés sempre té FIFO entre emissor i receptor. Llavors podem dir que encara és correcte l'afirmació.

4 → Els missatges entre diferents servidors pot divergir, ja que cada servidor processa els missatges que arriben via broadcast dels altres Servers en l'ordre d'arribada, no tenim un control general.

Podem dir que no tenim un ordre global, únicament podem mantenir un ordre local/parcial a cada servidor (tolerància estàtica i no dinàmica).

5 → Com bé hem dit anteriorment, no podem tenir una coherència en l'ordre de recepció global entre missatges de diferents servidors, és a dir, no hi ha garantia d'ordre consistent entre el contacte de clients de diferents servidors.

Clients locals poden veure el mateix ordre però no es pot generalitzar per la comunicació entre clients globals (de diferents servers).

Q3 - What happens if there are concurrent requests from servers to join or leave the system?

Si tenim sol·licituds simultànies de diferents servidors no podem gestiona-les alhora. Cada servidor farà broadcast de l'esdeveniment (leave or join) amb la llista actualitzada d'aquell moment. Per aquesta raó poden aparèixer data races, és a dir, llistes de Servers diferents de cada servidor que realitza l'esdeveniment alhora, no tenim cap protocol per la sincronització ni consistència d'esdeveniments al mateix temps.

Q4 - What are the advantages and disadvantages of this implementation regarding the previous one?

Per una banda, en quant a disponibilitat la versió 1 (servidor únic) si el servidor cau tot el sistema també ho fa, no tenim cap tolerància a les caigudes. En canvi, amb la segona versió tenim altres servidors disponibles per continuar donant el servei, tenim una tolerància parcial a les caigudes ja que no s'atura tot el sistema però els clients connectats al que ha caigut queden desconnectats.

Per altre banda, amb un servidor únicament tenim un ordre de missatges consistent i complet per tots els clients però no obtenim escalabilitat, ja que aquest servidor rep totes les peticions i si hi han moltíssims usuaris es pot saturar. En canvi, amb diferents servidors tenim un ordre local d'esdeveniments (pot ser diferent entre servidors) però amb una gran millora perquè els clients es poden distribuir entre diferents servidors amb la mateixa comunicació global amb l'inconvenient de la complexitat entre la coordinació d'aquests servidors.